

Java Backend Architecture

Interview Series

Chapter 15.8

API Versioning Strategies

50+ Interview Q&A

Code Examples

Architecture Diagrams

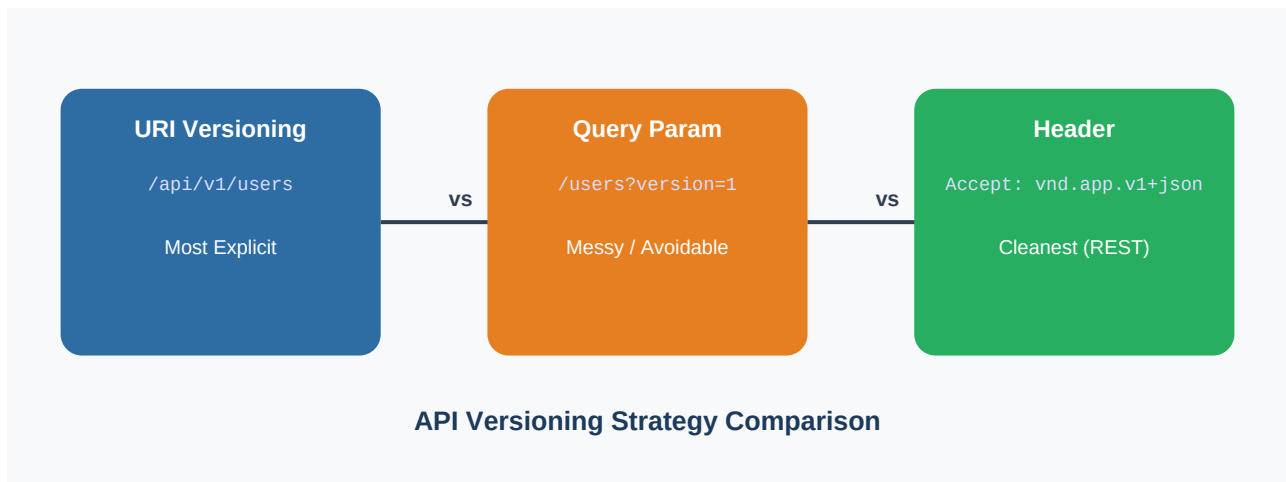
Advanced Topics

Chapter 15.8 – API Versioning Strategies

Overview

API versioning is essential for evolving microservices without breaking existing consumers. Three primary strategies exist—URI-based, query parameter, and header-based—each with distinct trade-offs. Combined with consumer-driven contract testing via Pact and formal deprecation via RFC 8594 Sunset headers, teams can evolve APIs safely at scale.

Versioning Strategies Comparison



URI Versioning

- Pattern: `/api/v1/users`, `/api/v2/users`
- Most explicit and cache-friendly; easily discoverable
- Breaks REST principle: same resource, different URLs
- Widely used (Twitter, GitHub, Stripe)

```
GET /api/v1/users/123 GET /api/v2/users/123 // new contract
```

Query Parameter Versioning

- Pattern: `/users?version=1` or `/users?v=2`
- Optional parameter makes contract implicit and messy
- Harder to cache; version easily forgotten
- Not recommended for public APIs

Header Versioning (Content Negotiation)

- Pattern: `Accept: application/vnd.myapp.v1+json`
- Cleanest approach—URI remains stable, version in Accept header
- Less discoverable; not visible in browser/curl by default
- Preferred by REST purists and API-first teams

```
Accept: application/vnd.myapp.v1+json Content-Type: application/vnd.myapp.v1+json
```

Spring Boot — Content Negotiation Config

```
@Configuration public class WebConfig implements WebMvcConfigurer { @Override public void configureContentNegotiation(ContentNegotiationConfigurer c) { c.defaultContentType(MediaType.APPLICATION_JSON) .mediaType("v1", MediaType.parseMediaType("application/vnd.myapp.v1+json")) .mediaType("v2", MediaType.parseMediaType("application/vnd.myapp.v2+json")); } }
```

Backward Compatibility Rules

- Safe changes (additive): adding new fields, optional query params, new endpoints
- Breaking changes: removing/renaming fields, changing types, removing endpoints
- Use `@JsonIgnoreProperties(ignoreUnknown = true)` on all DTOs
- Never change the meaning of an existing field
- Semantic versioning: MAJOR.MINOR.PATCH — bump MAJOR on breaking changes

```
@JsonIgnoreProperties(ignoreUnknown = true) public class UserDto { private String id; private String name; // New optional fields added in v2 – backward compatible private String email; // null-safe for v1 consumers }
```

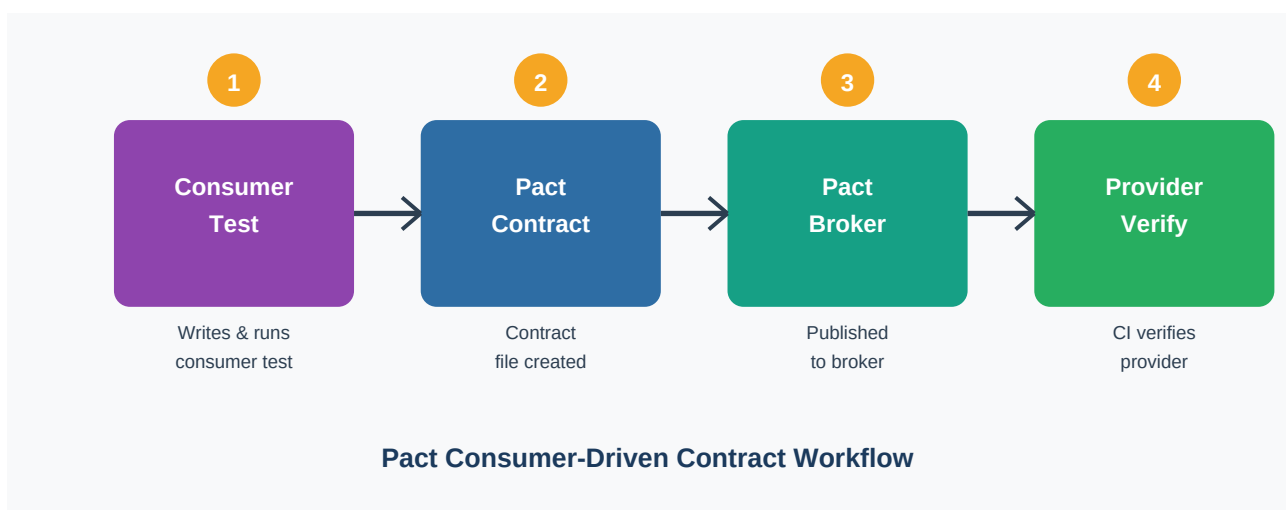
Deprecation Headers (RFC 8594)

RFC 8594 defines the **Sunset** header to formally signal when an API version will be deactivated. Combine with the **Deprecation** header (IETF draft) and Link headers for migration docs.

```
HTTP/1.1 200 OK Deprecation: true Sunset: Sat, 31 Dec 2025 23:59:59 GMT Link: <https://api.example.com/migration>; rel="successor-version" Content-Type: application/json
```

- Spring Filter injects Deprecation + Sunset on deprecated routes
- Clients should log/alert on Deprecation header presence
- Sunset date must be communicated at least 6 months ahead

Pact: Consumer-Driven Contract Testing

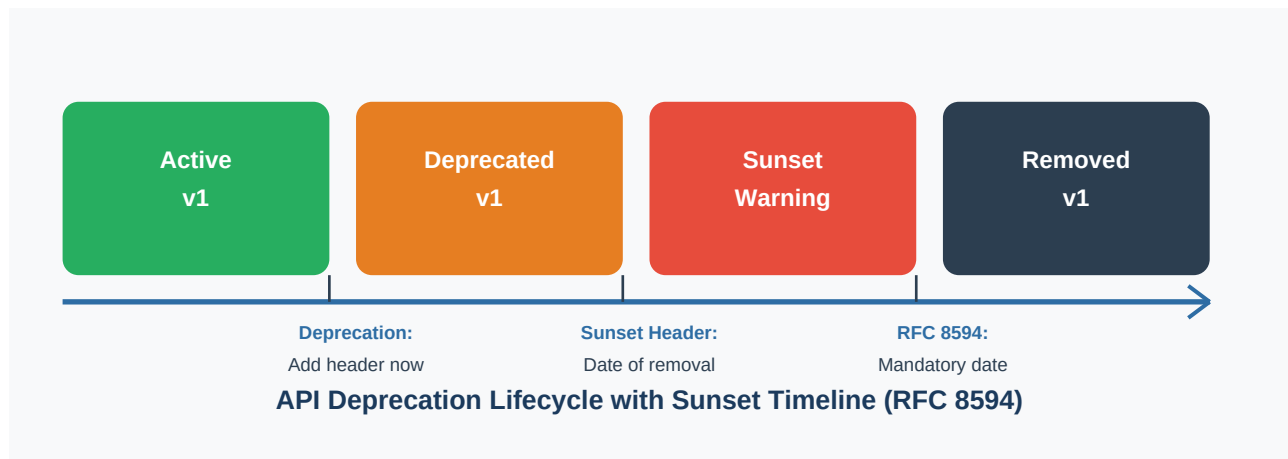


Pact prevents providers from inadvertently breaking consumers. The consumer writes an interaction test, Pact generates a contract (JSON), which is published to a Pact Broker. The provider verifies the contract in CI.

```
// Consumer test (JVM Pact) @ExtendWith(PactConsumerTestExt.class) class UserClientPactTest {
@Pact(consumer = "order-service", provider = "user-service") RequestResponsePact
createPact(PactDslWithProvider builder) { return builder .given("user 123 exists")
.uponReceiving("GET user by id") .path("/users/123").method("GET")
.willRespondWith().status(200) .body(newJsonBody(b -> b.stringType("name", "Alice")).build())
.toPact(); } }
```

```
// Provider verification (Spring Boot Test) @Provider("user-service") @PactBroker(host =
"pact-broker", port = "9292") @SpringBootTest(webEnvironment = RANDOM_PORT) class
UserServicePactProviderTest { @TestTarget public final Target target = new
HttpTarget("localhost", port); // Runs all consumer contracts automatically from broker }
```

API Deprecation Lifecycle



Versioning Strategy Comparison Table

Strategy	Discoverability	REST Purity	Caching	Recommended For
URI /api/v1/	High	Low	Excellent	Public APIs, SDKs
Query ?version=1	Medium	Low	Poor	Internal, legacy
Accept Header	Low	High	Good	REST-first, enterprise
Hypermedia (HATEOAS)	Self-describing	Highest	Good	Mature REST APIs

Interview Q&A — Chapter 15.8: API Versioning Strategies

Q1. What are the three main API versioning strategies?

URI versioning (/api/v1/users) — most explicit, widely supported, breaks REST resource concept. Query parameter versioning (/users?version=1) — implicit, messy, hard to cache. Header versioning (Accept: application/vnd.myapp.v1+json) — cleanest REST approach, version in content negotiation header, less discoverable.

Q2. Why does URI versioning break the REST resource concept?

REST resources should have stable, canonical URIs. Adding a version prefix means the same logical resource (/users/123) exists at multiple URIs (/api/v1/users/123 and /api/v2/users/123). Despite this, URI versioning is practically the most common due to its simplicity and cache-friendliness.

Q3. What is header-based versioning and why is it considered the cleanest approach?

Header versioning uses the HTTP Accept header for content negotiation: Accept: application/vnd.myapp.v1+json. It's cleanest because the URI remains stable—representing the same resource—while the version is metadata about the representation format. This aligns perfectly with REST principles. The downside is reduced discoverability.

Q4. What constitutes a backward-compatible (non-breaking) API change?

Safe/additive changes: adding new optional fields to responses, adding new optional request parameters, adding new endpoints, adding new enum values (with caution), relaxing validation. Breaking changes: removing fields, renaming fields, changing field types, making optional fields required, removing endpoints, changing HTTP method or status codes.

Q5. Why should all DTOs use @JsonIgnoreProperties(ignoreUnknown = true)?

When a provider adds new fields to a response, consumers without this annotation will fail with UnrecognizedPropertyException during deserialization. With ignoreUnknown=true, consumers safely ignore new fields they don't understand, maintaining backward compatibility as APIs evolve additively.

Q6. Explain semantic versioning for APIs.

Semantic versioning (SemVer): MAJOR.MINOR.PATCH. For APIs: MAJOR increments on breaking changes (existing clients must update). MINOR increments on backward-compatible new features. PATCH increments on backward-compatible bug fixes. API versioning typically exposes only MAJOR in the version (/v1/, /v2/) since minor/patch changes should be transparent to consumers.

Q7. What is the Deprecation header and when should it be added?

The Deprecation header (IETF draft) signals that an API version is being phased out. It should be added as soon as a version is marked for deprecation—before the Sunset date. Example: Deprecation: true. Clients monitoring this header can alert their teams to plan migration.

Q8. What does RFC 8594 define and what is the Sunset header?

RFC 8594 defines the Sunset header to formally communicate when an API endpoint will be decommissioned. Sunset: Sat, 31 Dec 2025 23:59:59 GMT specifies the HTTP-date after which the resource will not be available. Optionally, a Link header can point to migration documentation or the successor version URI.

Q9. How do you implement versioning in Spring Boot using RequestMappingHandlerMapping?

Custom HandlerMapping can inspect headers or URI prefixes to route to different controller versions. Simpler: use content negotiation configuration (`WebMvcConfigurer.configureContentNegotiation`) mapping media types to version identifiers, then annotate controllers with `produces = 'application/vnd.app.v1+json'`. Spring selects the handler matching the Accept header.

Q10. What is consumer-driven contract testing?

The consumer (e.g., order-service) defines the API interactions it expects—request/response pairs—in a contract file. The provider (e.g., user-service) must verify it satisfies all consumer contracts in CI. This ensures providers don't accidentally break consumers. Pact is the most widely used framework for this.

Q11. Describe the Pact workflow end-to-end.

1. Consumer writes an interaction test using Pact DSL, which generates a JSON contract file. 2. Contract is published to the Pact Broker (central contract repository). 3. Provider's CI pipeline pulls contracts from the broker using `@PactBroker` annotation. 4. Provider runs `@Provider` verification tests replaying each consumer interaction against the real provider. 5. Results are published back to the broker. 'Can I deploy?' checks prevent deployment if contracts are broken.

Q12. What is the Pact Broker and why is it needed?

The Pact Broker is a shared service that stores contracts, tracks verification results, and enables 'can-i-deploy' queries. Without it, contract files must be manually shared between teams. The broker provides a UI showing which consumer versions are compatible with which provider versions, supports tagging (main, prod), and integrates with CI/CD pipelines.

Q13. How does Pact prevent false positives in contract testing?

Pact uses a mock provider on the consumer side—the consumer test runs against a local mock, not the real service. The contract captures only what the consumer actually uses, not the full API. Provider verification replays each interaction against the real provider. This means providers can add fields freely; only removals/renames of fields the consumer uses will fail.

Q14. What is the difference between contract testing and integration testing?

Integration tests spin up both services and test against real implementations—slow, brittle, environment-dependent. Contract tests decouple consumer and provider testing: consumer tests run against a mock; provider tests verify contracts independently. Contract tests are fast, run in isolation, and can run in CI without other services running.

Q15. How do you handle versioning for internal vs external APIs differently?

External/public APIs: strict versioning (URI-based for discoverability), long support windows (12-24 months), formal deprecation with Sunset headers, SDKs must support multiple versions. Internal APIs: can use header versioning or even trunk-based evolution with contract testing; coordination between teams via Pact broker eliminates need for long support windows.

Q16. What is hypermedia-based versioning (HATEOAS)?

HATEOAS (Hypermedia As The Engine Of Application State) embeds version-aware links in responses. The client navigates the API by following links rather than constructing URIs. Version changes are expressed via different rel types or Content-Types in link headers. This is the most REST-pure approach but requires significant client sophistication and is rarely used in practice.

Q17. What is the 'Tolerant Reader' pattern?

The Tolerant Reader pattern states that API consumers should only extract the fields they need and ignore everything else. Combined with `@JsonIgnoreProperties(ignoreUnknown=true)`, consumers remain functional as providers add new fields. This reduces tight coupling between consumer and provider schema versions.

Q18. How should you version GraphQL APIs?

GraphQL is designed to avoid versioning: add new fields/types without breaking existing queries, use `@deprecated` on fields with a deprecation reason, and use schema evolution rather than versioning. If major breaking changes are needed, create a new schema (versionless endpoint) rather than versioning the URL. Introspection lets clients discover deprecated fields.

Q19. What are the risks of removing an API version too quickly?

Consumers may have missed deprecation notices; mobile clients have long upgrade cycles. Hard-coded URLs in client code are often missed. Third-party integrations may not monitor Deprecation headers. Best practice: monitor traffic to deprecated endpoints, send direct notifications to API key holders, enforce a minimum 6-12 month sunset period after deprecation announcement.

Q20. How do you implement a Deprecation filter in Spring Boot?

Implement `OncePerRequestFilter` or `HandlerInterceptor`. In the filter, check if the request path matches a deprecated version prefix (e.g., `/api/v1/`). If so, add response headers: `Deprecation: true`, `Sunset: [RFC 1123 date]`, and `Link: [migration URL]`. Register the filter with `@Component` or `FilterRegistrationBean` for applicable URL patterns.

Q21. What is 'API evolution' vs 'API versioning'?

API evolution means designing APIs to be extensible without requiring version changes—using optional fields, flexible schemas, content negotiation, and Tolerant Reader pattern. API versioning is a fallback for when evolution isn't possible (breaking changes). Good API design delays the need for versioning through careful use of additive patterns.

Q22. How does Spring Boot's `@RequestMapping` 'produces' attribute support versioning?

Controllers annotated with `produces = 'application/vnd.myapp.v1+json'` are matched only when the client sends `Accept: application/vnd.myapp.v1+json`. Different controller classes or methods can handle different versions via `produces`. Spring's `RequestMappingHandlerMapping` ranks handlers by media type specificity, selecting the best match.

Q23. What is 'version negotiation' and how does it work?

Version negotiation lets clients request a version range rather than a specific version. The server returns the best match. In HTTP: `Accept: application/vnd.app+json; version=2` or multiple `Accept` values with q-factors (quality values). The server responds with `Content-Type` indicating the actual version served, allowing gradual client migration.

Q24. How do you test API versioning in Spring Boot?

Use `MockMvc` or `WebTestClient` with explicit `Accept` headers: `mockMvc.perform(get('/users/1').header('Accept','application/vnd.app.v1+json'))`. Test that v1 endpoints return v1 schema, v2 returns v2 schema. Test that deprecated endpoints return `Deprecation` and `Sunset` headers. Use Pact for consumer-driven contract tests across versions.

Q25. What is 'API key versioning' and when is it appropriate?

Each API key is bound to a specific API version at creation time. New keys get the latest version; existing keys continue on their version. When rotating to a new key, the client migrates to the new version.

Appropriate for B2B integrations where clients have long release cycles; ensures no surprise breaking changes.

Q26. How does the 'Strangler Fig' pattern apply to API versioning?

When rewriting an API, rather than releasing a big-bang v2, the Strangler Fig pattern routes requests incrementally to the new implementation: first new endpoints, then migrated endpoints. A facade/proxy routes v1 requests to old or new implementation feature-by-feature. Eventually, v1 is completely served by the new implementation, and the old code is removed.

Q27. What are common pitfalls in URI versioning?

Forgetting to version all related resources (versioning /users but not /orders). Maintaining version indefinitely without deprecation strategy. Duplicating code between versions instead of using a shared service layer. Deep version nesting (/api/v2/users/v3/orders). Not communicating sunset dates, leaving consumers on old versions.

Q28. How do you manage database schema changes with API versioning?

Database schema must support both versions during transition. Strategies: expand-and-contract (add new column, migrate data, update v2 to use it, remove old column after v1 sunsets). Version-specific DTOs map from the same underlying entity. Database migration tools (Flyway, Liquibase) manage schema evolution in a backward-compatible way.

Q29. What is the 'can-i-deploy' feature in Pact?

'Can I deploy?' is a Pact Broker CLI/API query that checks whether a specific version of a service is compatible with all currently deployed consumer/provider versions. It prevents deploying a provider that breaks any consumer contract. Integrated into CI/CD as a deployment gate: `pact-broker can-i-deploy --participant user-service --version 1.2.3 --to production`.

Q30. What is bidirectional contract testing in Pact?

Pact's bidirectional contract testing (PactFlow feature) allows using existing API specs (OpenAPI) rather than writing Pact tests. The provider uploads an OpenAPI spec; consumers upload their expectations. The broker checks compatibility without requiring the provider to run Pact verification tests. This reduces the effort needed for providers to adopt Pact.

Q31. How do you handle versioning for event-driven APIs (Kafka)?

Use Schema Registry (Confluent) with Avro/Protobuf schemas. Schema evolution rules: BACKWARD compatibility — new schema can read old messages. FORWARD compatibility — old schema can read new messages. FULL compatibility — both directions. Schema Registry rejects schema changes that violate the configured compatibility level. Consumer groups use schema IDs in message headers to deserialize correctly.

Q32. What are 'vendor media types' in API versioning?

Vendor media types (vnd.) follow IANA conventions: `application/vnd.company.resource.v1+json`. Format: `application/vnd.{org}.{type}.v{version}+{format}`. They allow content negotiation for versioning while following HTTP standards. The +json suffix indicates JSON encoding, allowing parsers to handle unknown subtypes as JSON.

Q33. How do you document deprecated APIs effectively?

In OpenAPI/Swagger: set `deprecated: true` on operation or parameter, add description with sunset date and migration path. In Javadoc: `@Deprecated` annotation with `@see` pointing to replacement. In developer portal: banner or badge on deprecated endpoints, migration guide, code examples for new version. Automated:

generate deprecation reports from annotations in CI.

Q34. What is the role of API gateways in version management?

API gateways (Kong, AWS API Gateway, Azure APIM) centralize version routing: Route `/api/v1/` to v1 service, `/api/v2/` to v2 service. Strip version prefix before passing to backend. Add Deprecation/Sunset headers globally for deprecated routes. Enforce version-specific rate limits and authentication. Collect per-version usage metrics to inform sunset decisions.

Q35. How does Spring Boot's `@Deprecated` interact with API versioning?

`@Deprecated` on a controller method marks it for IDE warnings and deprecation tools. Combine with a custom `@ApiVersion` annotation and AOP aspect that injects Deprecation response headers. OpenAPI documentation generation tools (springdoc-openapi) read `@Deprecated` and mark the operation deprecated in the spec. This creates a consistent deprecation signal across code, docs, and HTTP headers.

Q36. What is 'version pinning' in microservices consumers?

Version pinning means a consumer explicitly specifies which API version it uses in its configuration rather than using 'latest'. This prevents unexpected behavior when a provider releases a new version. Best practice: pin in service discovery/config, use contract tests to verify the pinned version is still valid, and have a process for deliberately upgrading the pin.

Q37. Explain how to migrate from URI versioning to header versioning.

Transition strategy: 1. Add header versioning support alongside existing URI versioning. 2. Document and encourage clients to migrate to header-based. 3. Monitor header vs URI usage via logging. 4. After sufficient migration (e.g., 90% header usage), deprecate URI versioning with Sunset header. 5. Eventually remove URI routing. This requires dual-mode routing during transition.

Q38. What is API contract testing vs API schema testing?

Contract testing (Pact): verifies that specific consumer interactions work against the provider — consumer-specific, tests actual usage patterns. Schema testing (OpenAPI/JSON Schema): verifies that responses conform to the declared API schema — provider-centric, tests completeness of implementation against specification. Both are complementary: schema tests catch regressions, contract tests catch consumer-specific breaks.

Q39. How do you handle multiple API versions in a single Spring Boot application?

Option 1: Controller per version — `V1UserController` and `V2UserController` with different produces. Option 2: Service layer abstraction — controllers delegate to versioned service interfaces. Option 3: Feature flags — single controller, conditional logic based on version header. Best practice: use Option 1 for major versions to keep code clean; share service/repo layer across versions.

Q40. What is the relationship between API versioning and microservice independence?

Proper API versioning enables independent deployability: services can be deployed independently without coordinating release windows. The consumer pinning a contract version ensures it works with any provider version that satisfies the contract. Without versioning, all services must be deployed together (distributed monolith). Contract testing (Pact) verifies compatibility without deployment coordination.

Q41. What monitoring should you implement for versioned APIs?

Track: request count and error rate per API version (deprecation dashboard). Alert when deprecated version usage exceeds threshold (consumers still using it). Track response time per version (performance regression detection). Log which consumer applications are using which version (from User-Agent or API key). Set up

automated alerts 30/60/90 days before Sunset date.

Q42. How do you ensure backward compatibility in Protobuf/gRPC APIs?

Protobuf rules: never change field numbers, never change field types, new required fields break compatibility (use optional), never rename fields (field numbers matter, not names), remove fields by marking reserved (reserved 5; reserved 'old_field'). gRPC versioning: use package versioning (service UserServiceV2), or field_behavior annotations for deprecation.

Q43. What is the 'expand-contract' pattern for API migrations?

Expand phase: add new field/endpoint alongside old one, deploy, notify consumers. Migrate phase: consumers update to use new field/endpoint, both active simultaneously. Contract phase: remove old field/endpoint after all consumers have migrated. This zero-downtime pattern ensures no consumer breakage. The Deprecation/Sunset headers signal the contract phase start date during the migrate phase.

Q44. How do you version APIs in a BFF (Backend for Frontend) architecture?

BFF absorbs versioning complexity: mobile BFF and web BFF independently version their APIs. The BFF translates between its stable frontend API and the evolving backend microservice APIs. Frontend consumers see a stable BFF API; BFF upgrades backend API versions independently. This reduces the number of versioned consumers for each microservice.

Q45. What is API versioning in the context of event-driven architecture?

Event schemas must also be versioned. Breaking event schema changes require: publishing new event type (UserCreatedV2) alongside old (UserCreated), updating producers to emit both, migrating consumers to new event type, deprecating old event type (monitoring subscriber count), removing old event type after all consumers migrated. Schema Registry enforces compatibility rules automatically for Avro/Protobuf.

Q46. What is 'API lifecycle management'?

API lifecycle management covers: Design (OpenAPI spec first), Development (implement against spec), Testing (contract tests, schema validation), Publishing (developer portal, versioning), Deprecation (Deprecation/Sunset headers, migration guides), Retirement (monitoring, final removal). Platforms like Apigee, Kong Gateway, or AWS API Gateway provide lifecycle management features including automated Sunset enforcement.

Q47. What is the 'API-first' development approach and how does it relate to versioning?

API-first means the OpenAPI specification is written before implementation. Versioning is designed explicitly in the spec from the start—including media types, deprecation markers, and extension points for future fields. Contract tests are generated from the spec. This prevents ad-hoc versioning decisions and ensures the API is designed for evolvability from day one.

Q48. How do you handle versioning for pagination and filtering parameters?

Additive: new filter parameters are backward compatible (optional params). Breaking: changing pagination from page+size to cursor-based requires a new version. Strategy: introduce cursor alongside page-based in v1 with Deprecation notice on page param. Sunset page-based in v2. For filter parameters, use strict schema validation — unknown filter params return 400 in strict mode or are ignored in tolerant mode.

Q49. What is 'semantic versioning for REST APIs' and its limitations?

Applying SemVer to REST APIs: v1.2.3 where major=breaking, minor=new features, patch=bug fixes. Limitation: exposing all three in URLs (/api/v1.2.3/) is impractical and creates too many versions. Common practice: expose only major version in URI (/api/v1/), communicate minor/patch in documentation. Changelog

and API update newsletters cover minor/patch changes without requiring URL changes.

Q50. What is the difference between REST API versioning and SDK versioning?

REST API versioning controls the server-side contract; SDK versioning controls the client library. An SDK version may support multiple API versions (backward compatibility in the SDK). SDK v2.0 might call both API v1 and v2 during transition. The SDK changelog communicates which API versions are supported and when older API support is dropped. API and SDK release cycles are typically independent.

Q51. How do you enforce API versioning policies organizationally?

Technical enforcement: API gateways reject requests without version, linting rules require deprecation headers, CI checks that OpenAPI specs include deprecation markers before merge. Process enforcement: architecture review board approves breaking changes, API registry tracks all versions and sunset dates, developer portal displays version status prominently. Pact 'can-i-deploy' gates prevent deploying breaking changes to production.

Q52. What is a 'version facade' pattern?

A version facade is a thin translation layer that maps v1 requests/responses to the v2 internal model. Rather than maintaining two full codebases, the facade handles field renames, type conversions, and structural differences. V1 facade → V2 service → V2 domain model. This reduces code duplication and ensures the v1 facade is the only code that changes when v2 evolves. Trade-off: increased complexity in the facade layer.

Q53. How do you test Pact contracts in a CI/CD pipeline?

Consumer CI: run Pact consumer tests, publish generated contract to Pact Broker with build number/branch tag. Provider CI: pull consumer contracts from broker (@PactBroker), run provider verification tests, publish verification results back to broker. Deployment gates: before deploying consumer, run 'can-i-deploy' against provider version in target env. Before deploying provider, run 'can-i-deploy' against all consumer contracts for that environment.

Q54. What is the impact of microservices versioning on service discovery?

Service discovery (Consul, Kubernetes services) must support version routing. Options: separate service names per version (user-service-v1, user-service-v2) — clear but operationally complex. Or single service name with version tags and gateway routing. Kubernetes: separate Deployment + Service per version with ingress routing by path or header. This enables canary deployments — gradually shifting traffic from v1 to v2.

Q55. What is the 'tolerant provider' pattern?

The tolerant provider accepts and ignores unexpected request fields rather than returning 400. Allows consumers to evolve independently — a consumer sending new fields to an old provider won't break. Complementary to Tolerant Reader pattern on the consumer side. Implementation: use @JsonIgnoreProperties(ignoreUnknown=true) on request DTOs too. Caveat: security consideration — unexpected fields should be logged and monitored.